

GameBoyGame

Technical Documentation

Maxime ENTZ / Enzo LAUGEL / Mélanie SCHMITT

May 16, 2026

Contents

1	Project Overview	3
1.1	Game Concept and Target Hardware	3
1.2	Repository Structure	3
1.3	Toolchain	4
1.4	Build and Run From Scratch	5
2	Code Conventions	6
2.1	Naming Conventions	6
2.2	File Organization Rules	6
2.3	Comment Style	7
2.4	Hardware-Specific Coding Rules	7
3	Hardware Reference	9
3.1	Memory Map Used by the Project	9
3.2	Hardware Registers	10
3.3	Interrupt Handlers	10
4	Architecture and Module Documentation	11
4.1	Overall Architecture	11
4.2	Main Module	11
4.2.1	Source Files	11
4.3	Common Modules	12
4.3.1	Source Files and State	12
4.4	Menu Modules	16
4.4.1	Source Files and State	16
4.5	Lobby Modules	18
4.5.1	Source Files and State	18
4.6	Minigame 1: Trap Memory	20
4.6.1	Source Files and State	20
4.7	Minigame 2: Tetris	22
4.7.1	Source Files and State	22
4.8	Minigame 3: Flappy Bird	25
4.8.1	Source Files and State	25
4.9	Asset Modules	26
5	Data Formats	27
5.1	Tile and Sprite Format	27
5.2	Current VRAM Tile Allocation	27
5.3	Tilemap Format	27
5.4	Audio Data	28
5.5	Save Data	28

6	Step-by-Step Guide: Adding a Minigame	30
6.1	1. Create the Source and Header	30
6.2	2. Register It in the Build System	30
6.3	3. Add a Game State	31
6.4	4. Make It Reachable	31
6.5	5. Allocate VRAM Without Conflicts	32
6.6	6. Load and Display a Tilemap	32
6.7	7. Read Joypad Input	32
6.8	8. Handle VBlank Correctly	33
6.9	9. Return to the Main Flow	33
6.10	10. Minimal Working Skeleton	33

Chapter 1

Project Overview

1.1 Game Concept and Target Hardware

GameBoyGame is a Game Boy homebrew game built around a lobby and several minigames. The player starts from a menu, selects or creates a save, chooses a difficulty, explores a tile-based island lobby, talks to NPCs through a dialogue box, and is routed into minigames such as Trap Memory, Tetris, and Flappy Bird.

The build target is a DMG-compatible `.gb` ROM. The Makefile configures a cartridge with MBC5, RAM, and battery-backed save support using `-Wm-yt27`, with eight 16 KiB ROM banks using `-Wm-yo8`, and one 8 KiB external RAM bank using `-Wm-ya1`. The code uses DMG-style palettes and does not contain CGB palette or double-speed configuration, so it should be treated as a DMG target that also runs on GBC or GBA in compatibility mode.

1.2 Repository Structure

`asset/` Generated or hand-maintained C asset arrays for tiles, sprites, and tilemaps.

`asset/include/` Public headers for asset arrays. These headers expose `extern const UINT8` declarations and tile ID enums.

`asset/include/common/` Shared font and dialogue-box asset declarations.

`asset/include/menu/` Menu and save-select screen asset declarations.

`asset/include/mg1/` Trap Memory-specific asset declarations.

`asset/include/mg2/` Tetris tile and background asset declarations.

`asset/include/mg3/` Flappy Bird tile and background asset declarations.

`asset/src/` Asset array definitions compiled directly into the ROM.

`asset/src/common/` Shared font and dialogue-box tile definitions.

`asset/src/menu/` Menu and save-select tilemap or tile definitions.

`asset/src/mg1/` Trap Memory safe-tile definitions.

`asset/src/mg2/` Tetris tileset and background tilemap definitions.

`asset/src/mg3/` Flappy Bird tileset and background tilemap definitions.

`include/` Public project headers.

`include/common/` Shared engine-style services: common types, text rendering, dialogue, transitions, RNG, save data, VRAM clearing, and tile lookup.

`include/lobby/` Lobby state, map transition, lore, and scoreboard interfaces.

`include/main/` Core loop and scene-change interface.

`include/menu/` Start menu, save-select, and difficulty-select interfaces.

`include/mg1/` Trap Memory public interface and state definition.

`include/mg2/` Tetris state, grid, piece, HUD, and layout interfaces.

`include/mg3/` Flappy Bird public interface and layout constants.

`src/` Game logic source files.

`src/common/` Shared runtime modules.

`src/lobby/` Lobby loading, input, map switching, transition animation, lore flow, and scoreboard rendering.

`src/main/` Program entry point and core state machine.

`src/menu/` Menu, save-select, and difficulty-select scenes.

`src/mg1/` Trap Memory minigame.

`src/mg2/` Tetris minigame.

`src/mg3/` Flappy Bird minigame.

`docs/` Technical documentation, Doxygen configuration, and changelog.

`.github/workflows/` Repository automation configuration.

`.vscode/` Editor configuration.

`Makefile` Build, clean, rebuild, and Doxygen targets.

`README.md` Basic user-facing build instructions.

`game.sav` Local emulator save RAM file.

1.3 Toolchain

Compiler GBDK-2020 `lcc`, configured by `GBDK_HOME` `?= /opt/gbdk` and invoked as `$(GBDK_HOME)/bin/lcc`.

Assembler and linker Invoked through `lcc`. The Makefile passes `-Wa-l` for assembler listing output and `-Wl-m -Wl-j` for linker map and related output.

Cartridge configuration `-Wm-yt27 -Wm-yo8 -Wm-ya1`, meaning MBC5 + RAM + battery, 8 ROM banks, and 1 RAM bank according to the Makefile comments.

Asset pipeline Assets are C arrays in `asset/src/` with declarations in `asset/include/`. There is no conversion script, source image, or build rule in the repository.

Documentation tool Doxygen through the `make doc` target.

Emulator used for testing The repository only says to use a Game Boy emulator; it does not name one.

1.4 Build and Run From Scratch

Listing 1.1: Build from a fresh machine

```
1 # 1. Install make and GBDK-2020.
2 # On macOS, if a formula is available:
3 brew install gbdk-2020
4
5 # Otherwise download GBDK-2020 and point GBDK_HOME at it.
6 export GBDK_HOME="$HOME/gbdk"
7 export PATH="$GBDK_HOME/bin:$PATH"
8
9 # 2. Clone the repository.
10 git clone <repository-url> GameBoyGame
11 cd GameBoyGame
12
13 # 3. Build the ROM.
14 make
15
16 # 4. The output ROM is:
17 ls -l game.gb
```

The Makefile assumes `GBDK_HOME=/opt/gbdk` if the variable is not provided. Override it in the environment when GBDK is installed elsewhere:

Listing 1.2: Build with an explicit GBDK path

```
1 GBDK_HOME="$HOME/gbdk" make
```

Run `game.gb` in a DMG-compatible emulator.

Listing 1.3: Maintenance commands

```
1 make clean      # remove intermediate build files
2 make fclean     # remove intermediates and game.gb
3 make re         # clean rebuild
4 make doc        # run Doxygen using docs/Doxyfile
```

Chapter 2

Code Conventions

2.1 Naming Conventions

The codebase uses C with EPITECH-style file headers, include guards, and Doxygen-like comments. Real examples from the repository show the following conventions.

Functions Lowercase `snake_case`: `change_scene`, `text_renderer_draw`, `game_changer`, `save_write`, `grid_clear_lines`. Scene loader functions may be noun-like, such as `menu`, `lobby`, `tetris`, `trap_memory`, and `flappybird`.

Input handlers Scene input handlers use `handle_input_<scene>`, for example `handle_input_lobby`, `handle_input_tetris`, and `handle_input_save_select`.

Update handlers Scene update handlers use `update_<scene>`, for example `update_lobby`, `update_tetris`, and `update_flappybird`.

Static globals Static globals use the `g_` prefix: `g_state_function`, `g_lobby`, `g_tm`, `g_tetris`, `g_dialogue_ready`.

Types Typedef names end in `_t`: `game_t`, `lobby_state_t`, `trap_memory_t`, `piece_t`. Struct and enum tags end in `_s` and `_e`: `struct game_s`, `enum piece_type_e`.

Constants and macros Uppercase with underscores: `COMMON_SCREEN_WIDTH_TILES`, `MOVING_SENS_UP`, `SAVE_MAGIC`, `TETRIS_GRID_W`, `MG3_PIPE_GAP`.

Enums Enum values use uppercase prefixes that identify the domain: `GAME_STATE_MG1`, `MAP_ID_BL`, `DIFFICULTY_HARD`, `PIECE_T`, `LORE_ESCAPE`.

Asset symbols Asset arrays are lowercase descriptive names, such as `menu_bg_tiles`, `tetris_bg_map`, `player_tiles_front`, and `dialogue_box_tiles`.

2.2 File Organization Rules

`src/main/` Owns the entry point and the global state-machine loop.

`src/common/` Contains reusable services that do not belong to one scene: rendering text, dialogue boxes, transitions, save RAM, RNG, tile lookup, and VRAM clearing.

`src/menu/` Contains menu-like scenes that run before the lobby.

`src/lobby/` Contains the overworld/lobby state, map switching, player movement, dialogue progression, and scoreboard.

src/mgN/ Contains minigame N. Current minigames are **mg1** Trap Memory, **mg2** Tetris, and **mg3** Flappy Bird.

include/ Mirrors source ownership and exposes only cross-module types and functions.

asset/src/ and **asset/include/** Store ROM assets as C arrays. Logic files include asset headers but do not define asset data inline.

The Makefile compiles every `.c` file under **src/** and **asset/src/**, so adding a source file under those trees automatically includes it in the link. A new public interface still needs a matching header under **include/** and includes from the state table or caller.

2.3 Comment Style

Files begin with a block header:

Listing 2.1: File header style

```

1  /*
2  ** EPITECH PROJECT, 2026
3  ** GameBoyGame
4  ** File description:
5  ** core.c
6  */

```

Most public functions and many static helpers have Doxygen-style comments with **@brief**, **@param**, and sometimes **@return**. The common parameter marker macros **IN**, **OUT**, and **INOUT** are documentation aids defined as empty macros in **include/common/project_types.h**.

Some comments are in French, especially in the VBlank placeholder and a few TODO-like notes, but the dominant API documentation language is English.

2.4 Hardware-Specific Coding Rules

Frame pacing The core loop waits for VBlank with **wait_vbl_done()** before changing scenes, reading input, and updating the active scene.

VBlank handler **src/main/core.c** registers **vbl_interrupt** with **add_VBL**. The handler is intentionally empty and comments warn that only short work should be done there.

VRAM writes Scenes load tiles and tilemaps during scene setup, with display disabled by **change_scene**. Several update paths also call **set_bkg_tile_xy** or **set_bkg_tiles**; these should remain small enough for the frame budget.

Sprite budget The Game Boy has 40 OAM sprite entries. The project uses OAM indices 0–3 for the 16x16 lobby and Trap Memory player composed from four 8x8 sprites, and index 0 for the Flappy bird.

Tile budget DMG background tile IDs are one byte. Shared font tiles load at tile 128; dialogue box tiles load at tile 171. Scene-specific tiles loaded into lower tile IDs must not overlap shared text/dialogue tiles when text or dialogue is visible.

External RAM Save data is read and written only after **ENABLE_RAM** and before **DISABLE_RAM**. The save starts at **\$A000**.

ROM banking The Makefile configures 8 ROM banks, but the C code does not contain explicit **SWITCH_ROM** calls.

HRAM routines No project-defined **HRAM** routine or DMA trampoline is present in the source.

Cycle budgets No function-level cycle counts are documented.

Chapter 3

Hardware Reference

This chapter lists only hardware areas and registers that the project touches directly or through GBDK API calls present in the code.

3.1 Memory Map Used by the Project

Range	Size	Name	Project role
\$0000-\$3FFF	16 KiB	ROM0	Fixed ROM area generated by GBDK and <code>lcc</code> .
\$4000-\$7FFF	16 KiB	ROMX	Switchable MBC5 ROM banks; Makefile reserves 8 banks.
\$8000-\$97FF	6 KiB	VRAM tile data	Background, window, and sprite tile patterns loaded by <code>set_bkg_data</code> and <code>set_sprite_data</code> .
\$9800-\$9BFF	1 KiB	BG map 0	Visible 20x18 tilemaps written by <code>set_bkg_tiles</code> , <code>set_win_tiles</code> , and <code>set_bkg_tile_xy</code> .
\$9C00-\$9FFF	1 KiB	BG map 1	Available through GBDK; no direct project selection is found.
\$A000-\$BFFF	8 KiB	External SRAM	Battery-backed save area. <code>save_data_t</code> starts at \$A000.
\$C000-\$DFFF	8 KiB	WRAM	C globals, static module state, stack, and runtime data such as <code>g_lobby</code> , <code>g_tm</code> , <code>g_tetris</code> , and <code>tetris_grid</code> .
\$FE00-\$FE9F	160 B	OAM	Sprite attributes changed by <code>move_sprite</code> and <code>set_sprite_tile</code> .
\$FF00	1 B	JOYP	Joypad state read through <code>joypad()</code> .
\$FF04	1 B	DIV	RNG seed source through <code>DIV_REG</code> .
\$FF10-\$FF14	5 B	APU channel 1	Transition and dialogue sound effects.
\$FF24-\$FF26	3 B	APU control	Master volume, routing, and sound enable.
\$FF40	1 B	LCDC	Display, sprite, background, and window enables through GBDK macros.
\$FF47	1 B	BGP	DMG background palette used by fade transitions.
\$FF4A-\$FF4B	2 B	WY/WX	Window position changed by <code>move_win</code> .
\$FFFF	1 B	IE	VBlank interrupt enable handled by GBDK after <code>add_VBL</code> and <code>enable_interrupts</code> .

Table 3.1: Memory ranges used by GameBoyGame

3.2 Hardware Registers

Address	Register	Access	Bits and project use
\$FF00	JOYP/P1	Read	joypad() returns J_LEFT, J_RIGHT, J_UP, J_DOWN, J_A, J_B, J_START, and J_SELECT.
\$FF04	DIV	Read	random_init multiplies DIV_REG by the first nonzero key mask to seed initrand.
\$FF10	NR10	Write	0x00; channel 1 sweep disabled.
\$FF11	NR11	Write	0x81; channel 1 duty/length value for short effects.
\$FF12	NR12	Write	0x43; initial volume 4, decreasing envelope, period 3.
\$FF13	NR13	Write	0x90 for dialogue typing, or 0x30, 0x50, 0x70, 0x90 for transition tones.
\$FF14	NR14	Write	0x86; trigger bit set and frequency high bits set to 6.
\$FF24	NR50	Write	0xFF; max left/right output volume and VIN bits enabled.
\$FF25	NR51	Write	0xFF; all channels routed to both outputs.
\$FF26	NR52	Write	0x80; sound controller enabled.
\$FF40	LCDC	Write	Through DISPLAY_ON/OFF, SHOW/HIDE_SPRITES, SHOW/HIDE_BKG, and SHOW/HIDE_WIN; uses LCD enable bit 7, window enable bit 5, sprite enable bit 1, and background/window enable bit 0.
\$FF47	BGP	Write	Fade palettes 0xE4, 0xF9, 0xFE, 0xFF.
\$FF4A	WY	Write	move_win(MINWNDPOSX, DIALOGUE_WIN_Y) positions the dialogue window at pixel Y 112.
\$FF4B	WX	Write	move_win positions the dialogue window using MINWNDPOSX.

Table 3.2: Hardware registers touched by the source

GBDK functions such as `set_bkg_data`, `set_bkg_tiles`, `set_sprite_data`, `set_sprite_tile`, and `move_sprite` also write VRAM or OAM hardware areas. The project does not write the raw VRAM or OAM addresses manually.

3.3 Interrupt Handlers

VBlank Registered in core with `add_VBL(vbl_interrupt)`. Interrupts are enabled with `enable_interrupts()`

The handler body is empty and contains comments stating it is called each frame when the PPU is in VBlank and should remain short.

Other interrupts No LCD STAT, Timer, Serial, or Joypad ISR is registered by project code.

Cycle cost No cycle count is documented for `vbl_interrupt`.

Chapter 4

Architecture and Module Documentation

4.1 Overall Architecture

The program starts in `main`, calls `core`, initializes shared text and dialogue assets, registers an empty VBlank ISR, loads the initial menu, and then runs a frame loop. Each frame waits for VBlank, changes scene if requested, reads the joypad, dispatches input to the current scene, and dispatches the current scene update.

The scene table in `src/main/core.c` maps `game_state_t` values to three callbacks: load assets, handle input, and update. Implemented states are `GAME_STATE_MENU`, `GAME_STATE_SAVE_SELECT`, `GAME_STATE_CHOOSE_DIFFICULTY`, `GAME_STATE_LOBBY`, `GAME_STATE_MG1`, `GAME_STATE_MG2`, and `GAME_STATE_MG3`.

Cycle costs are not documented in the codebase.

4.2 Main Module

4.2.1 Source Files

`src/main/main.c` C entry point; calls `core`.

`src/main/core.c` Global scene table, display toggles, VBlank registration, input dispatch, update dispatch, and main loop.

main

Input None.

Output None.

Side effects Enters the game by calling `core`.

Calls `core`.

core

Input None.

Output None.

Side effects Initializes `game_t`, loads shared VRAM assets, installs the VBlank ISR, enables interrupts, controls LCD visibility, and runs until `game.is_running` becomes false.

Calls `text_renderer_init`, `dialogue_init`, `add_VBL`, `enable_interrupts`, `wait_vbl_done`, `change_scene`, scene input callbacks, scene update callbacks.

`change_scene`

Input `game` points to the global runtime state. `game->state_changed` determines whether work is done.

Output Clears `game->state_changed` after loading the new state.

Side effects Turns display, sprites, background, and window off; calls the current state's asset loader; then turns display, sprites, and background on.

Calls Current scene load callback.

4.3 Common Modules

4.3.1 Source Files and State

`src/common/text_renderer.c` Shared text renderer using the common font at tile base 128. Owns `g_text_renderer_ready`.

`src/common/dialogue.c` Dialogue box state machine and typing sound. Owns `g_dialogue_ready`.

`src/common/game_changer.c` State transitions with fade, sound, optional VRAM clearing, and immediate scene load.

`src/common/save.c` Battery-backed save load, write, and reset at `$A000`.

`src/common/transition.c` Palette fade and channel 1 transition sound effects.

`src/common/random.c` RNG seeding and modulo random helper. Owns `g_seed`.

`src/common/get_tile_by_map.c` Pixel-to-tile lookup helper for 20x18 maps.

`src/common/vram_clear.c` Clears 256 background tiles, background map, text/dialogue readiness flags, and all 40 sprites.

`text_renderer_init`

Input None.

Output None.

Side effects Loads `font_tiles` to VRAM starting at tile 128; marks renderer ready.

Calls `set_bkg_data`.

`text_renderer_destroy`

Input None.

Output None.

Side effects Allows the font tiles to be reloaded after VRAM clearing.

Calls None.

`text_renderer_draw`

Input `render` points to `text_render_t`; `text` must be null-terminated; `x` and `y` are tile coordinates.

Output None.

Side effects Writes font tile IDs to the background map until the string ends or the row reaches tile X 20.

Calls `set_bkg_tile_xy`.

`text_renderer_clear`

Input Same contract as `text_renderer_draw`.

Output None.

Side effects Replaces the rendered text range with the space tile.

Calls `set_bkg_tile_xy`.

`dialogue_init`

Input None.

Output None.

Side effects Loads dialogue box tiles into VRAM at tile 171; marks dialogue assets ready.

Calls `set_bkg_data`.

`dialogue_destroy`

Input None.

Output None.

Side effects Allows dialogue tiles to be reloaded after VRAM clearing.

Calls None.

`dialogue_start`

Input `dlg` points to a `dialogue_t`; `text` is a null-terminated string where `\n` separates lines.

Output Initializes the dialogue state to page 0 and animation state.

Side effects Writes the dialogue window tilemap, moves the window to Y 112, and shows the window.

Calls `set_win_tiles`, `move_win`, `SHOW_WIN`.

dialogue_update

Input `dlg` points to active dialogue state.

Output Advances typing timers and state.

Side effects Writes one character tile at a time to the window map and plays channel 1 typing sound for non-space characters.

Calls `set_win_tile_xy`; internal sound helper writes `NR10_REG` through `NR14_REG`, `NR50_REG`, `NR51_REG`, and `NR52_REG`.

dialogue_handle_input

Input `dlg` points to dialogue state; `keys` is the current `joypad()` mask.

Output May skip animation, advance a page, or close the dialogue.

Side effects Updates `dlg`; may hide the window.

Calls Internal page helpers and `HIDE_WIN`.

dialogue_is_active

Input `dlg` points to dialogue state.

Output Returns true when the dialogue state is not inactive.

Side effects None.

Calls None.

game_changer

Input `game` points to runtime state; `new_state` is the requested state; `clear_vram` decides whether to clear VRAM.

Output Updates `game->state` and `game->state_changed` if valid.

Side effects Plays transition sound, fades out and in, optionally clears VRAM, loads the new scene immediately via `change_scene`, and delays 300 ms.

Calls `transition_sound`, `transition_fade_out`, `vram_clear`, `change_scene`, `delay`, `transition_fade_in`.

save_load

Input `game` points to runtime state. SRAM must contain `save_data_t` or an invalid magic value.

Output Updates scores, lobby position, lobby map, dialogue index, and difficulty if `SAVE_MAGIC` matches.

Side effects Enables and disables external RAM; reads from `$A000`.

Calls `lobby_get_state`, `set_map`, `memcpy`.

save_write

Input `game` points to runtime state; lobby state is available through `lobby_get_state`.

Output Updates best scores in `game` if current scores are higher.

Side effects Writes `save_data_t` to external RAM at \$A000.

Calls `lobby_get_state`, `memcpy`.

save_reset

Input `game` provides current best scores.

Output None.

Side effects Writes a fresh save at \$A000, preserving best scores and resetting run scores, dialogue index, player position, map, and difficulty.

Calls `memcpy`.

transition_fade_out

Input None.

Output None.

Side effects Writes BGP_REG through four palettes from normal to dark and plays transition tones.

Calls `wait_vbl_done`, `transition_sound`, `delay`.

transition_fade_in

Input None.

Output None.

Side effects Writes BGP_REG through four palettes from dark to normal and plays transition tones.

Calls `wait_vbl_done`, `transition_sound`, `delay`.

transition_sound

Input `stage` selects one of four NR13 frequency low values; invalid stage resets to 0.

Output None.

Side effects Enables sound and triggers channel 1.

Calls None.

random_init

Input `keys` is the current nonzero joypad mask.

Output Seeds GBDK's RNG.

Side effects Reads DIV_REG; writes static `g_seed`.

Calls `initrand`.

`random_get`

Input `modulo` is used as the divisor for `rand() % modulo`; callers must pass nonzero.

Output Returns a `UINT8` in the range 0 to `modulo - 1`.

Side effects Advances the GBDK RNG state.

Calls `rand`.

`get_tile_by_map`

Input `map` is a 20x18 tile array; `x` and `y` are pixel coordinates.

Output Returns the tile ID at `x / 8, y / 8`; returns 0 out of bounds.

Side effects None.

Calls None.

`vram_clear`

Input None.

Output None.

Side effects Clears 256 background tile patterns to zero, invalidates shared text/dialogue loaded flags, fills the visible background map with tile 0, and moves all 40 sprites to 0,0.

Calls `set_bkg_data`, `text_renderer_destroy`, `dialogue_destroy`, `fill_bkg_rect`, `move_sprite`.

4.4 Menu Modules

4.4.1 Source Files and State

`src/menu/menu.c` Start screen loader and Start-button transition to save selection.

`src/menu/save_select.c` Continue/new-save menu. Owns cursor render state, save presence flag, and selected row.

`src/menu/difficult_select.c` Difficulty selection scene. Owns selected difficulty, key-release gating, and last key mask.

`menu`

Input `game` is unused.

Output None.

Side effects Loads `menu_bg_tiles` and `menu_bg_map`.

Calls `set_bkg_data`, `set_bkg_tiles`.

`handle_input_menu`

Input `game`; keys from `joypad()`.

Output None.

Side effects On `J_START`, switches to `GAME_STATE_SAVE_SELECT`.

Calls `game_changer`.

`update_menu`

Input `game` is unused.

Output None.

Side effects None.

Calls None.

`save_select`

Input `game` is unused.

Output None.

Side effects Loads save-select background, checks SRAM magic, draws CONTINUE, NEW SAVE, and cursor.

Calls `set_bkg_data`, `set_bkg_tiles`, `text_renderer_init`, `text_renderer_draw`, `memcpy`.

`handle_input_save_select`

Input `game`; keys from `joypad()`.

Output None.

Side effects Moves cursor; may load an existing save and enter lobby, or reset save and enter difficulty selection.

Calls `text_renderer_clear`, `text_renderer_draw`, `save_load`, `save_reset`, `game_changer`.

`update_save_select`

Input `game` is unused.

Output None.

Side effects None.

Calls None.

`choose_difficulty`

Input `game` is unused.

Output None.

Side effects Loads the difficulty menu background and draws labels and cursor; resets selection to easy and waits for key release.

Calls `set_bkg_data`, `set_bkg_tiles`, `text_renderer_init`, `text_renderer_draw`.

`handle_input_choose_difficulty`

Input `game`; keys from `joypad()`.

Output Sets `game->difficulty` when confirmed.

Side effects Updates cursor, writes selected difficulty into the save, and switches to lobby.

Calls `text_renderer_clear`, `text_renderer_draw`, internal SRAM writer, `game_changer`.

`update_choose_difficulty`

Input `game` is unused.

Output None.

Side effects None.

Calls None.

4.5 Lobby Modules

4.5.1 Source Files and State

`src/lobby/lobby.c` Lobby runtime state, asset loading, player sprite update, NPC animation, and frame counters. Owns static `g_lobby`.

`src/lobby/input_handler_lobby.c` Movement, collision, NPC interaction, RNG initialization, save-on-select, and menu return.

`src/lobby/map_manager_lobby.c` 3x3 map lookup, current map copy, and boundary map switching.

`src/lobby/transition_map_lobby.c` Blocking transition animation between maps.

`src/lobby/lore_lobby.c` Dialogue script and story progression. Owns `g_dialogue_texts` and idle text.

`src/lobby/scoreboard_lobby.c` Scoreboard text rendering.

`lobby_init_state`

Input None.

Output Initializes `g_lobby` to spawn position, map `MAP_ID_BL`, default movement direction, counters, and dialogue state.

Side effects Copies map data through `set_map`.

Calls `set_map`.

`lobby_get_state`

Input None.

Output Returns pointer to static `g_lobby`.

Side effects None.

Calls None.

lobby

Input `game` points to runtime state.

Output May load save data into `game` and lobby state the first time.

Side effects Loads lobby tiles, sprite data, current map, text/dialogue assets, and scoreboard if on `MAP_ID_TC`.

Calls `lobby_init_state`, `save_load`, `set_bkg_data`, `set_bkg_tiles`, `set_sprite_data`, `set_sprite_tile`, `move_sprite`, `text_renderer_init`, `dialogue_init`, `draw_scoreboard`.

handle_input_lobby

Input `game`; keys from `joypad()`.

Output Updates lobby player position and movement direction.

Side effects Seeds RNG on first key press, advances dialogue input, starts NPC dialogue with A, writes save and returns to menu on Select, and may switch maps.

Calls `random_init`, `dialogue_is_active`, `dialogue_handle_input`, `lore_start_dialogue`, `lore_start_idle`, `save_write`, `game_changer`, `get_tile_by_map`, `switch_map`.

update_lobby

Input `game` points to runtime state.

Output Updates lobby counters and sprite position.

Side effects Animates NPC tiles, updates lore/dialogue, updates four player sprites.

Calls `lore_update`, `set_bkg_data`, `set_sprite_tile`, `move_sprite`.

transition_map_animation

Input `game`; `dir` is `TRANSITION_TB`, `TRANSITION_BT`, `TRANSITION_LR`, or `TRANSITION_RL`.

Output Updates lobby player coordinates after crossing the map boundary.

Side effects Displays `transition_map`, moves player sprites in a blocking loop, waits for VBlank per step, then restores the new current map and scoreboard if needed.

Calls `set_bkg_tiles`, `move_sprite`, `wait_vbl_done`, `draw_scoreboard`.

switch_map

Input `game`; `x` and `y` are pixel collision coordinates; `tile` is the touched tile.

Output Returns true if a map switch starts.

Side effects Updates current map ID and map copy, marks map as changing, and runs transition animation.

Calls `transition_map_animation`.

`set_map`

Input lobby; map_id.

Output Sets lobby->current_map_id and copies 360 tile IDs into lobby->current_map.

Side effects Invalid IDs fall back to MAP_ID_BL.

Calls None.

`lore_start_idle_dialogue`

Input None.

Output None.

Side effects Starts idle dialogue without advancing lore.

Calls lobby_get_state, dialogue_is_active, dialogue_start.

`lore_start_dialogue`

Input game points to runtime state for score and difficulty checks.

Output None.

Side effects May adjust dialogue_index to a win branch and starts the current lore dialogue.

Calls lobby_get_state, dialogue_is_active, dialogue_start.

`lore_update`

Input game points to runtime state.

Output None.

Side effects Auto-starts story dialogue, updates dialogue animation, transitions to minigames or menu when story steps finish, writes or resets save data.

Calls dialogue_update, dialogue_is_active, game_changer, save_write, save_reset.

`draw_scoreboard`

Input game supplies best and current scores.

Output None.

Side effects Draws scoreboard labels and zero-padded score rows onto the background map.

Calls text_renderer_draw.

4.6 Minigame 1: Trap Memory

4.6.1 Source Files and State

`src/mg1/trap_memory.c` Trap Memory map, HUD, round timing, safe tile generation, scoring, game-over handling, and sprite placement. Owns `g_tm`.

`src/mg1/tm_handle_input.c` Trap Memory movement and collision.

`src/mg1/tm_sprite_handle.c` Trap Memory player sprite animation.

trap_memory

Input game points to runtime state.

Output Resets game->score_mg1 to 0 and game->level to 1.

Side effects Initializes g_tm, loads lobby-style terrain and safe tiles, writes the Trap Memory tilemap, hides window, initializes text renderer, and draws HUD.

Calls set_bkg_data, set_bkg_tiles, set_sprite_tile, HIDE_WIN, text_renderer_init.

trap_memory_get

Input None.

Output Returns pointer to static g_tm.

Side effects None.

Calls None.

handle_input_trap_memory

Input game; keys from joypad().

Output Updates Trap Memory player position and movement direction.

Side effects Does nothing while block_movement is true; otherwise moves by SPEED if collision allows.

Calls trap_memory_get, get_tile_by_map.

update_trap_memory

Input game points to runtime state.

Output Updates score, level, round counters, and player sprite positions.

Side effects Shows safe tiles, hides them after TIME_SEE_SAFE_TILE, detects game over, writes victory/game-over text, and returns to lobby when finished.

Calls text_renderer_draw, random_get, set_bkg_tiles, game_changer, move_sprite_personage, move_sprite.

move_sprite_personage

Input game points to trap_memory_t.

Output Clears game->is_moving.

Side effects Chooses four sprite tile IDs according to direction and animation frame.

Calls set_sprite_tile.

4.7 Minigame 2: Tetris

4.7.1 Source Files and State

src/mg2/tetris.c Tetris scene state, spawning, drop timing, scoring, level progression, and input. Owns `g_tetris`.

src/mg2/tetris_grid.c 10x18 playfield storage and line clearing. Owns global `tetris_grid`.

src/mg2/tetris_piece.c Piece shapes, drawing, erasing, movement checks, lock, spawn check, and rotation.

src/mg2/tetris_hud.c Score, level, line, and next-piece rendering.

tetris

Input `game` points to runtime state.

Output Resets `game->score_mg2`, `game->level`, and Tetris counters.

Side effects Initializes grid, hides first four sprites, loads Tetris tiles and background, draws HUD, spawns and draws the first piece.

Calls `random_get`, `grid_init`, `move_sprite`, `set_bkg_data`, `text_renderer_init`, `set_bkg_tiles`, `draw_hud`, `draw_next_piece`, `piece_draw`.

handle_input_tetris

Input `game`; keys from `joypad()`.

Output Updates current piece rotation, horizontal position, and soft-drop timer.

Side effects Select returns to lobby; A rotates right, B rotates left, Down accelerates drop, Left/Right move with throttle.

Calls `game_changer`, `rotate_r`, `rotate_l`, `piece_can_move_left`, `piece_can_move_right`, `piece_erase`, `piece_draw`.

update_tetris

Input `game` points to runtime state.

Output Advances current piece downward or locks it when blocked.

Side effects Updates background tiles, grid, score, level, lines, next preview, and may return to lobby on spawn failure.

Calls `piece_can_move_down`, `piece_erase`, `piece_draw`, `piece_lock`, `grid_clear_lines`, `grid_draw_playfi`, `draw_score`, `draw_lines`, `draw_level`, `draw_next_piece`, `piece_can_spawn`, `game_changer`.

grid_init

Input None.

Output Sets every cell in `tetris_grid` to 0.

Side effects Modifies `tetris_grid`.

Calls None.

`grid_clear_lines`

Input `tetris_grid` contains tile IDs or 0.

Output Returns number of cleared lines.

Side effects Collapses full rows in `tetris_grid`.

Calls Internal row helpers.

`grid_draw_playfield`

Input `tetris_grid`.

Output None.

Side effects Redraws the 10x18 playfield into the background map at X offset `MG2_PLAYFIELD_X`.

Calls `set_bkg_tile_xy`.

`piece_draw`

Input `piece` points to a valid `piece_t`.

Output None.

Side effects Draws occupied 4x4 shape cells to the background map.

Calls `set_bkg_tile_xy`.

`piece_erase`

Input `piece` points to a valid `piece_t`.

Output None.

Side effects Replaces occupied piece cells with `TETRIS_TILE_EMPTY`.

Calls `set_bkg_tile_xy`.

`piece_can_move_down`

Input `piece` points to a valid `piece_t`.

Output Returns true if the shape can move down inside bounds and without grid collision.

Side effects None.

Calls None.

`piece_can_move_left`

Input `piece`.

Output Returns true if the piece fits one cell left.

Side effects None.

Calls Internal fit helper.

`piece_can_move_right`

Input `piece`.

Output Returns true if the piece fits one cell right.

Side effects None.

Calls Internal fit helper.

`piece_lock`

Input `piece`.

Output None.

Side effects Writes occupied cells into `tetris_grid`.

Calls None.

`piece_can_spawn`

Input `piece`.

Output Returns true if the piece fits at its current spawn position.

Side effects None.

Calls Internal fit helper.

`rotate_r`

Input `piece`.

Output Updates `piece->rot` and possibly `piece->x`.

Side effects Erases and redraws the piece if rotation fits, including a one-cell wall adjustment.

Calls `piece_erase`, `piece_draw`.

`rotate_l`

Input `piece`.

Output Updates `piece->rot` and possibly `piece->x`.

Side effects Erases and redraws the piece if rotation fits, including a one-cell wall adjustment.

Calls `piece_erase`, `piece_draw`.

`draw_score`

Input `score`.

Output None.

Side effects Draws a six-digit zero-padded score.

Calls `text_renderer_draw`.

`draw_lines`

Input lines.

Output None.

Side effects Draws a three-digit zero-padded line counter.

Calls `text_renderer_draw`.

`draw_level`

Input level.

Output None.

Side effects Draws a two-digit zero-padded level value.

Calls `text_renderer_draw`.

`draw_hud`

Input score, level, and lines.

Output None.

Side effects Draws Tetris HUD labels and values.

Calls `text_renderer_draw`, `draw_score`, `draw_level`, `draw_lines`.

`draw_next_piece`

Input type is a `piece_type_t`.

Output None.

Side effects Draws the next-piece preview box interior.

Calls `set_bkg_tile_xy`.

4.8 Minigame 3: Flappy Bird

4.8.1 Source Files and State

`src/mg3/flappy.c` Flappy Bird state, pipe generation, collision, drawing, gravity, and jump input. Owns static `fbird`.

`flappybird`

Input game is unused.

Output Initializes bird Y and pipe positions.

Side effects Loads Flappy tiles and background, copies bird tile data into sprite tile 0, and assigns sprite 0.

Calls `random_get`, `set_bkg_data`, `set_bkg_tiles`, `set_sprite_data`, `set_sprite_tile`.

`handle_input_flappybird`

Input `game` is unused; `keys` from `joypad()`.

Output Updates bird Y on J_A.

Side effects Subtracts MG3_JUMP from `fbird.bird_y`.

Calls None.

`update_flappybird`

Input `game` points to runtime state.

Output Updates bird and pipe positions.

Side effects Applies gravity, moves sprite 0, draws and wraps pipes in the background map, and returns to lobby on collision or falling below the screen.

Calls `game_changer`, `move_sprite`, `set_bkg_tile_xy`, `random_get`.

4.9 Asset Modules

Asset source files contain public `const` `UINT8` arrays compiled into ROM. They do not own mutable state and do not call functions.

`asset/src/asset_lobby.c` Lobby terrain, player, NPC, transition, and 3x3 lobby map arrays.

`asset/src/common/asset_common_font.c` Common font tile array.

`asset/src/common/asset_dialogue.c` Dialogue box tile and window map arrays.

`asset/src/menu/asset_menu.c` Start menu tiles and tilemap.

`asset/src/menu/asset_save_select.c` Save-select background tilemap.

`asset/src/mg1/asset_tm.c` Trap Memory safe-tile overlays.

`asset/src/mg2/asset_tetris.c` Tetris tileset and background map.

`asset/src/mg3/asset_flappy.c` Flappy tileset and background map.

Chapter 5

Data Formats

5.1 Tile and Sprite Format

All graphics assets are C arrays of `UINT8`. Tile data follows the Game Boy 2bpp 8x8 tile format: 16 bytes per tile, two bytes per pixel row, with two bitplanes forming four DMG color indices. No compression or runtime decompression is present in the codebase.

Background tiles Loaded with `set_bkg_data(first_tile, count, data)`. Examples include lobby terrain at tile IDs 0–30, Tetris tiles at tile IDs 0–17, Flappy tiles at tile IDs 0–2, common font at tile ID 128, and dialogue box tiles at tile ID 171.

Sprite tiles Loaded with `set_sprite_data`. The lobby player uses 8x8 sprite mode and composes a 16x16 character from four sprites. Flappy Bird uses one 8x8 sprite for the bird.

Palette The project uses DMG palettes. Fade transitions write only `BGP_REG`; no CGB background or object palette data is present.

Compression None found.

5.2 Current VRAM Tile Allocation

Because each scene reloads tiles after `game_changer` can clear VRAM, tile IDs are scene-local except for shared text and dialogue systems. Any minigame that uses `text_renderer_draw` must avoid tile IDs 128–170. Any minigame that uses dialogue must also avoid 171–179.

5.3 Tilemap Format

Tilemaps are flat `UINT8` arrays of tile IDs. Visible screens are written as 20 columns by 18 rows, matching `COMMON_SCREEN_WIDTH_TILES` and `COMMON_SCREEN_HEIGHT_TILES`. The project writes these maps with:

Listing 5.1: Tilemap load pattern

```
1 set_bkg_tiles(0, 0, 20, 18, some_map);
```

The lobby stores nine maps named `map_tl`, `map_tc`, `map_tr`, `map_cl`, `map_cc`, `map_cr`, `map_bl`, `map_bc`, and `map_br`. Runtime lobby collision uses a mutable copy in `lobby_state_t.current_map`. Trap Memory similarly uses a mutable 20x18 map to reveal and hide safe tiles.

Tetris uses a separate logical grid:

tetris_grid A 10x18 `UINT8` matrix. Zero means empty; nonzero values are Tetris tile IDs.

Tile IDs	Owner	Loaded by	Notes
0-255	Menu background	menu	MENU_BG_TILE_COUNT is 256; fills the BG tile ID space.
0	Save/difficulty background	save_select, choose_difficulty	Loads only void_tile; text uses common font.
0-30	Lobby terrain/NPC tiles	lobby	Terrain and NPC animation tiles.
0-23	Lobby player sprite tiles	lobby	Sprite tile IDs for four-direction player animation.
0-23	Trap Memory player sprite tiles	Asset dependency from lobby player tiles	Trap Memory sets sprite tile IDs but does not call set_sprite_data locally.
90-91	Trap Memory safe overlays	trap_memory	OFFSET_SAFER_TILE plus two safe tile variants.
0-17	Tetris tiles	tetris	TETRIS_TILE_COUNT.
0-2	Flappy tiles	flappybird	Empty, pipe, bird. Bird is also copied into sprite tile 0.
128-170	Common font	text_renderer_init	FTILE_COUNT is 43.
171-179	Dialogue box	dialogue_init	DTILE_COUNT is 9.
180-255	Unreserved by shared systems	None	Available unless the active scene loads overlapping tiles.

Table 5.1: Observed VRAM tile ID usage

Screen placement Grid cells render at background X offset MG2_PLAYFIELD_X, with row 0 at screen tile Y 0.

Flappy Bird uses the background tilemap as mutable pipe columns. Pipe X is tracked in pixels and converted to tile columns; columns wrap through a 32-tile background width using MG3_BKG_WIDTH_SIZE.

5.4 Audio Data

There is no hUGEDriver, GBT Player, tracker export, song table, or SFX table in the repository. Audio is custom direct register programming for short channel 1 effects:

Dialogue typing dialogue_update triggers a short sound for each non-space character by writing NR50, NR51, NR52, and channel 1 registers NR10–NR14.

Transitions transition_sound writes the same channel 1 setup while varying NR13 by transition stage.

5.5 Save Data

The cartridge is configured with one 8 KiB external RAM bank. Save data starts at \$A000, defined as SRAM_BASE. The magic value SAVE_MAGIC is 0x1A71. The code enables RAM before memcpy and disables RAM afterward.

The exact storage width of difficulty_t is compiler-defined because it is a C enum.

Field	Type	Size	Role
magic	UINT16	2 B	Must equal SAVE_MAGIC.
best_mg1	UINT16	2 B	Best Trap Memory score.
best_mg2	UINT16	2 B	Best Tetris score.
best_mg3	UINT16	2 B	Best Flappy score.
run_mg1	UINT16	2 B	Current Trap Memory run score.
run_mg2	UINT16	2 B	Current Tetris run score.
run_mg3	UINT16	2 B	Current Flappy run score.
dialogue_index	UINT8	1 B	Current lore step.
player_x	UINT8	1 B	Lobby player sprite X coordinate.
player_y	UINT8	1 B	Lobby player sprite Y coordinate.
current_map_id	UINT8	1 B	Current lobby map ID.
difficulty	difficulty_t	implementation- defined	Selected difficulty.

Table 5.2: save_data_t layout at \$A000

Chapter 6

Step-by-Step Guide: Adding a Minigame

This guide shows how to add a fourth minigame using the current state-machine pattern. The example names the game `mg4` and follows the existing conventions.

6.1 1. Create the Source and Header

Create a new source directory and header directory:

Listing 6.1: Create module directories

```
1 mkdir -p src/mg4 include/mg4 asset/src/mg4 asset/include/mg4
```

Use lowercase `snake_case` file names. For example:

`include/mg4/mg4.h` Public scene loader, input handler, and update function.

`src/mg4/mg4.c` Runtime state and minigame logic.

`asset/include/mg4/asset_mg4.h` Tile and tilemap declarations.

`asset/src/mg4/asset_mg4.c` Tile and tilemap byte arrays.

6.2 2. Register It in the Build System

No Makefile source list edit is needed if the files are under `src/` and `asset/src/`. The Makefile uses:

Listing 6.2: Existing Makefile source discovery

```
1 SRC      = $(shell find src/ -name "*.c")
2 ASSET    = $(shell find asset/src/ -name "*.c")
```

Add include paths are already broad enough:

Listing 6.3: Existing include flags

```
1 -Iinclude/ -Iasset/include/
```

6.3 3. Add a Game State

Add a new value before `GAME_STATE_COUNT` in `include/common/common.h`:

Listing 6.4: Add a state value

```

1 typedef enum {
2     GAME_STATE_LOBBY,
3     GAME_STATE_PLAYING,
4     GAME_STATE_PAUSED,
5     GAME_STATE_VICTORY,
6     GAME_STATE_MG1,
7     GAME_STATE_MG2,
8     GAME_STATE_MG3,
9     GAME_STATE_MG4,
10    GAME_STATE_GAME_OVER,
11    GAME_STATE_MENU,
12    GAME_STATE_SAVE_SELECT,
13    GAME_STATE_CHOOSE_DIFFICULTY,
14    GAME_STATE_COUNT
15 } game_state_t;

```

Then include the new header and add the state to `g_state_function` in `src/main/core.c`:

Listing 6.5: Register scene callbacks

```

1 #include "mg4/mg4.h"
2
3 static const management_state_t g_state_function[MAX_STATES] = {
4     /* existing entries */
5     {GAME_STATE_MG4, mg4, handle_input_mg4, update_mg4},
6     /* existing entries */
7 };

```

6.4 4. Make It Reachable

The lobby currently starts minigames from `src/lobby/lore_lobby.c` after dialogue steps. To route to the new minigame, call `game_changer` with the new state from the appropriate story branch:

Listing 6.6: Enter the new minigame from lore

```

1 game_changer(game, GAME_STATE_MG4, TRUE);

```

For a temporary test hook, route an unused lobby input in `handle_input_lobby`; remove it before shipping:

Listing 6.7: Temporary debug entry point

```

1 if (keys & J_START) {
2     game_changer(game, GAME_STATE_MG4, TRUE);
3     return;
4 }

```

The current code already uses `J_SELECT` to save and return to the menu from the lobby, so avoid reusing Select for this hook.

Tile IDs	Owner	Use in a new minigame
0-127	Scene-local graphics	Best place for minigame tiles.
128-170	Common font	Do not overwrite if using <code>text_renderer_draw</code> .
171-179	Dialogue box	Do not overwrite if using dialogue.
180-255	Currently free shared range	Usable if the scene does not need many tiles; keep documented.

Table 6.1: Minigame-safe VRAM planning

6.5 5. Allocate VRAM Without Conflicts

The current shared VRAM map is:

For a normal minigame, load its tiles from tile 0 and keep the count below 128 if it uses text. If the minigame needs more than 128 background tiles and does not use the common text renderer, it may use more of the tile ID range after calling `vram_clear` through `game_changer(..., TRUE)`.

6.6 6. Load and Display a Tilemap

Use the same 20x18 tilemap convention as the other scenes:

Listing 6.8: Tile and tilemap loading

```

1 set_bkg_data(0, MG4_TILE_COUNT, mg4_tiles);
2 set_bkg_tiles(0, 0,
3             COMMON_SCREEN_WIDTH_TILES,
4             COMMON_SCREEN_HEIGHT_TILES,
5             mg4_bg_map);

```

The map array should contain 20 * 18 tile IDs unless you intentionally use scrolling or the full 32x32 Game Boy background map.

6.7 7. Read Joypad Input

Do not call `joypad()` in the minigame. The core loop already reads it once per frame in `handle_keys` and passes the mask to your scene input handler. Use the existing `J_*` masks:

Listing 6.9: Input handler pattern

```

1 void
2 handle_input_mg4(OUT game_t *game, IN UINT8 keys)
3 {
4     if (keys & J_SELECT) {
5         game_changer(game, GAME_STATE_LOBBY, TRUE);
6         return;
7     }
8     if (keys & J_A) {
9         /* action */
10    }
11 }

```

6.8 8. Handle VBlank Correctly

The project has one VBlank ISR registered in `core`; it is empty. A minigame should not install its own ISR unless the core architecture is changed. Keep minigame work in the scene callbacks:

Loader Bulk-load tiles and tilemaps in `mg4`. It is called with the display off during scene changes.

Input Consume the passed `keys` mask in `handle_input_mg4`.

Update Do small per-frame work in `update_mg4`. The core loop already calls it after `wait_vbl_done`.

Avoid long loops in `update_mg4`. If an animation must block like `transition_map_animation`, call `wait_vbl_done()` inside the loop.

6.9 9. Return to the Main Flow

Return to the lobby through the same transition helper used by current minigames:

Listing 6.10: Return to lobby

```
1 game_changer(game, GAME_STATE_LOBBY, TRUE);
```

If the minigame contributes to persistent progress, update the appropriate score fields before returning and call `save_write(game)` from the lobby/story flow or directly if the design requires immediate persistence.

6.10 10. Minimal Working Skeleton

Listing 6.11: include/mg4/mg4.h

```
1 #ifndef MG4_H_
2     #define MG4_H_
3
4     #include "common/common.h"
5     #include "common/game_changer.h"
6     #include "common/text_renderer.h"
7     #include "mg4/asset_mg4.h"
8
9 void
10 mg4(OUT game_t *game);
11
12 void
13 handle_input_mg4(OUT game_t *game, IN UINT8 keys);
14
15 void
16 update_mg4(OUT game_t *game);
17
18 #endif /* !MG4_H_ */
```

Listing 6.12: src/mg4/mg4.c

```
1 /*
2  ** EPITECH PROJECT, 2026
3  ** GameBoyGame
4  ** File description:
5  ** mg4.c
6  */
```

```
7
8 #include "mg4/mg4.h"
9
10 typedef struct mg4_state_s {
11     UINT8 frame;
12     BOOLEAN finished;
13 } mg4_state_t;
14
15 static mg4_state_t g_mg4;
16
17 void
18 mg4(OUT game_t *game)
19 {
20     (void)game;
21     g_mg4.frame = 0;
22     g_mg4.finished = FALSE;
23
24     set_bkg_data(0, MG4_TILE_COUNT, mg4_tiles);
25     set_bkg_tiles(0, 0,
26                  COMMON_SCREEN_WIDTH_TILES,
27                  COMMON_SCREEN_HEIGHT_TILES,
28                  mg4_bg_map);
29     text_renderer_init();
30 }
31
32 void
33 handle_input_mg4(OUT game_t *game, IN UINT8 keys)
34 {
35     if (keys & J_SELECT) {
36         game_changer(game, GAME_STATE_LOBBY, TRUE);
37         return;
38     }
39     if (keys & J_A) {
40         game->score_mg3++;
41     }
42 }
43
44 void
45 update_mg4(OUT game_t *game)
46 {
47     g_mg4.frame++;
48     if (g_mg4.finished) {
49         game_changer(game, GAME_STATE_LOBBY, TRUE);
50         return;
51     }
52 }
```

The skeleton intentionally uses real project functions: `text_renderer_init`, `game_changer`, and the core scene-callback signatures. Replace `game->score_mg3` with a new score field if the project adds persistent support for a fourth minigame.